

Identifiers in C Programming

Names for variables, functions, arrays, and more

C identifiers represent the name in the C program, for example, variables, functions, arrays, structures, unions, labels, etc. An identifier can be composed of letters such as uppercase, lowercase letters, underscore, digits, but the starting letter should be either an alphabet or an underscore.

Introduction to Identifiers



What are Identifiers?

We can say that an identifier is a collection of alphanumeric characters that begins either with an alphabetical character or an underscore, which are used to represent various programming elements such as variables, functions, arrays, structures, unions, labels, etc.



Character Set for Identifiers

There are 52 alphabetical characters (uppercase and lowercase), underscore character, and ten numerical digits (0-9) that represent the identifiers. There is a total of 63 alphanumeric characters that represent the identifiers.



Key Point

Identifiers are user-defined names that give unique identity to various elements in a C program.

2

Rules for Constructing C Identifiers



The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.



It should not begin with any numerical digit.



In identifiers, both uppercase and lowercase letters are distinct. Therefore, we can say that identifiers are case sensitive.



Commas or blank spaces cannot be specified within an identifier.



Keywords cannot be represented as an identifier.



The length of the identifiers should not be more than 31 characters.



Identifiers should be written in such a way that it is meaningful, short, and easy to read.

3

Examples of Identifiers



Valid Identifiers

Here are some examples of valid identifiers in C:

Total

Valid

sum

Valid

average

Valid

m

Valid

sum_1

Valid

studentName

Valid

calculateSum

Valid

MAX_VALUE

Valid



Invalid Identifiers

Here are some examples of invalid identifiers in C:

2sum

Invalid

int

Invalid

my var

Invalid

total, amount

Invalid

123

Invalid

for

Invalid

while

Invalid

case

Invalid

4

Types of Identifiers 🚀



Internal Identifier

If the identifier is not used in the external linkage, then it is known as an internal identifier. The internal identifiers can be local variables.

Example:

```
int main() {  
    int localVar = 10; // Internal identifier  
    // Function code here  
    return 0;  
}
```



External Identifier

If the identifier is used in the external linkage, then it is known as an external identifier. The external identifiers can be function names, global variables.

Example:

```
// In file1.c
int globalVar = 20; // External identifier
void externalFunction(); // External identifier

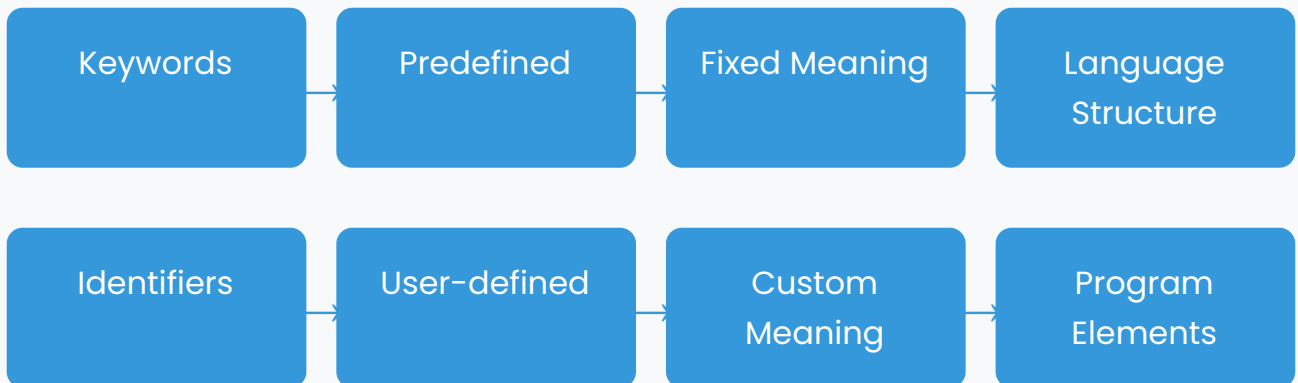
// In file2.c
extern int globalVar; // Reference to external identifier
extern void externalFunction(); // Reference to external identifier
```

5

Differences between Keyword and Identifier

Keyword	Identifier
Keyword is a pre-defined word.	The identifier is a user-defined word.
It must be written in a lowercase letter.	It can be written in both lowercase and uppercase letter.
Its meaning is pre-defined in the c compiler.	Its meaning is not defined in the c compiler.

Keyword	Identifier
It is a combination of alphabetical characters.	It is a combination of alphanumeric characters.
It does not contain the underscore character.	It can contain the underscore character.



6

Best Practices for Naming Identifiers



Be Descriptive: Use names that clearly describe the purpose of the variable or function.



Keep it Concise: While being descriptive, also keep names reasonably short.



Use Consistent Naming Conventions: Either use camelCase (studentName) or snake_case (student_name) consistently throughout your code.



Avoid Single-Letter Names: Except for loop counters (i, j, k), avoid using single-letter variable names.



Avoid Confusing Names: Don't use names that differ only in case (total and Total) or are easily confused (l and 1).



Follow Domain Conventions: When working on specific projects, follow the naming conventions established in that domain.

```
// Good naming examples
int studentCount = 25;
float averageScore = 85.5;
void calculateTotal(int scores[], int count);

// Bad naming examples
int sc = 25; // Not descriptive
float a = 85.5; // Not descriptive
void ct(int s[], int c); // Abbreviated
```

Conclusion 🎓

Character Sets in C Programming

Character sets in C programming are crucial for representing, storing, and processing characters within programs. Understanding character encoding schemes, escape sequences, handling wide characters, and addressing localization challenges are essential for

writing robust and portable C programs that effectively handle diverse character sets and language requirements.



Key Takeaways

- ASCII, the fundamental character encoding standard, provided a base for representing text characters in early computing.
- Unicode, a comprehensive standard, allows for the representation of a wide range of characters from different languages and scripts.
- Multibyte characters, particularly in UTF-8, enable the representation of characters beyond the ASCII range, facilitating multilingual support and text processing in modern computing environments.
- Locale functions allow C programs to adapt to users' language and cultural preferences, affecting formatting and text handling.
- Character set functions enable proper manipulation and classification of characters based on locale-specific rules, which is crucial for multilingual text processing.
- Internationalization is the foundational step in making software adaptable to diverse languages and cultures, enabling global reach and usability.

Identifiers are fundamental to C programming, providing names for variables, functions, and other program elements. By following proper naming conventions and rules, programmers can create clear, maintainable, and efficient code.